https://chatgpt.com/c/69ff17dc-b374-83aa-879c-c5c5171bef26

3 Histograms

Histograms are used to depict image statistics in an easily interpreted visual format. With a histogram, it is easy to determine certain types of problems in an image, for example, it is simple to conclude if an image is properly exposed by visual inspection of its histogram. In fact, histograms are so useful that modern digital cameras often provide a real-time histogram overlay on the viewfinder (Fig. 3.1) to help prevent taking poorly exposed pictures. It is important to catch errors like this at the image capture stage because poor exposure results in a permanent loss of information which it is not possible to recover later using image-processing techniques. In addition to their usefulness during image capture, histograms are also used later to improve the visual appearance of an image and as a "forensic" tool for determining what type of processing has previously been applied to an image.

3.1 What Is a Histogram?

Histograms in general are frequency distributions, and histograms of images describe the frequency of the intensity values that occur in an image. This concept can be easily explained by considering an old-fashioned grayscale image like the one shown in Fig. 3.2. A histogram h for a grayscale image I with intensity values in the range $I(u, v) \in [0, K-1]$ would contain exactly K entries, where for a typical 8 bit grayscale image, $K = 2^8 = 256$. Each individual histogram entry is defined as

$$\mathsf{h}(i) = \text{ the } \textit{number} \text{ of pixels in } I \text{ with the intensity value } i,$$

W. Burger, M.J. Burge, *Principles of Digital Image Processing*, Undergraduate Topics in Computer Science, DOI 10.1007/978-1-84800-191-6_3, © Springer-Verlag London Limited, 2009

Figure 3.1 Digital camera back display showing a histogram overlay.

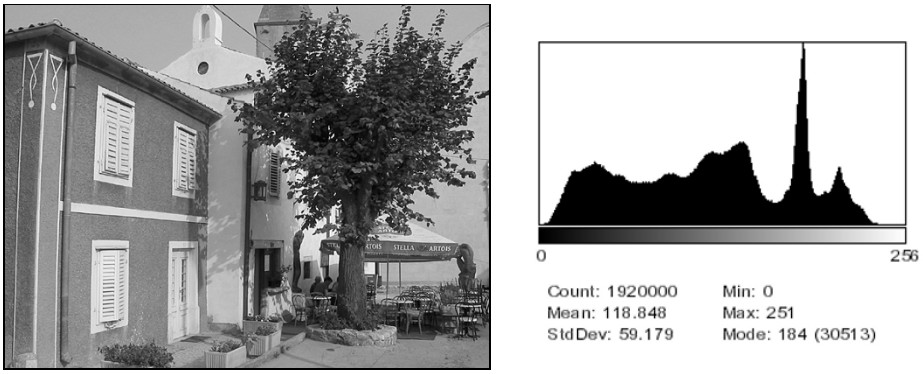


Figure 3.2 An 8-bit grayscale image and a histogram depicting the frequency distribution of its 256 intensity values.

for all $0 \leq i < K$. More formally stated,

$$\mathsf{h}(i) = \text{card}\big\{(u,v) \mid I(u,v) = i\big\}.^{1} \tag{3.1}$$

Therefore $\mathsf{h}(0)$ is the number of pixels with the value 0, $\mathsf{h}(1)$ the number of pixels with the value 1, and so forth. Finally $\mathsf{h}(255)$ is the number of all white pixels with the maximum intensity value $255 = K-1$. The result of the histogram computation is a one-dimensional vector h of length K. Figure 3.3 gives an example for an image with $K = 16$ possible intensity values.

Since a histogram encodes no information about *where* each of its individual entries originated in the image, histograms contain no information about the spatial arrangement of pixels in the image. This is intentional since the

[1] card$\{\ldots\}$ denotes the number of elements ("cardinality") in a set (see also p. 233).

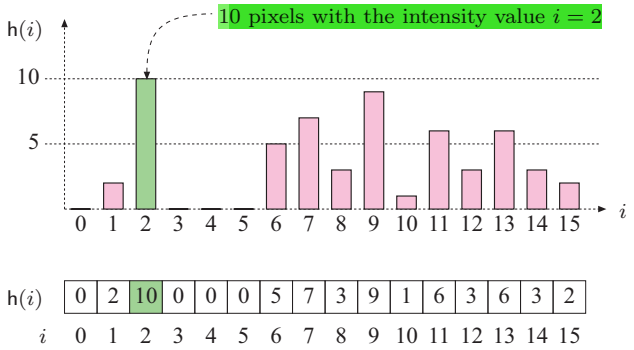


Figure 3.3 Histogram vector for an image with $K = 16$ possible intensity values. The indices of the vector element $i = 0 \ldots 15$ represent intensity values. The value of 10 at index 2 means that the image contains 10 pixels of intensity value 2.

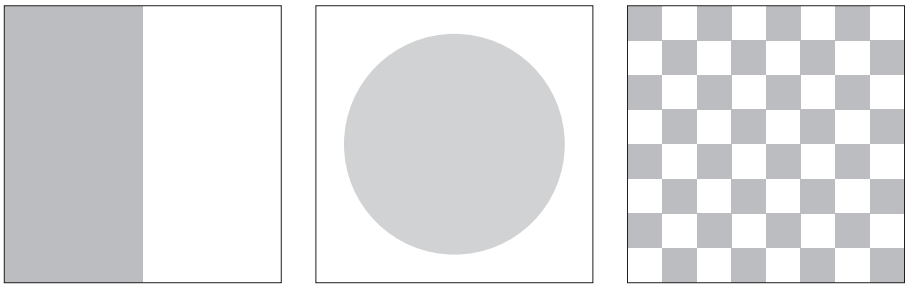


Figure 3.4 Three very different images with identical histograms.

main function of a histogram is to provide statistical information, (e.g., the distribution of intensity values) in a compact form. Is it possible to reconstruct an image using only its histogram? That is, can a histogram be somehow "inverted"? Given the loss of spatial information, in all but the most trivial cases, the answer is no. As an example, consider the wide variety of images you could construct using the same number of pixels of a specific value. These images would appear different but have exactly the same histogram (Fig. 3.4).

3.2 Interpreting Histograms

A histogram depicts problems that originate during image acquisition, such as those involving contrast and dynamic range, as well as artifacts resulting from image-processing steps that were applied to the image. Histograms are often used to determine if an image is making effective use of its intensity range (Fig. 3.5) by examining the size and uniformity of the histogram's distribution.

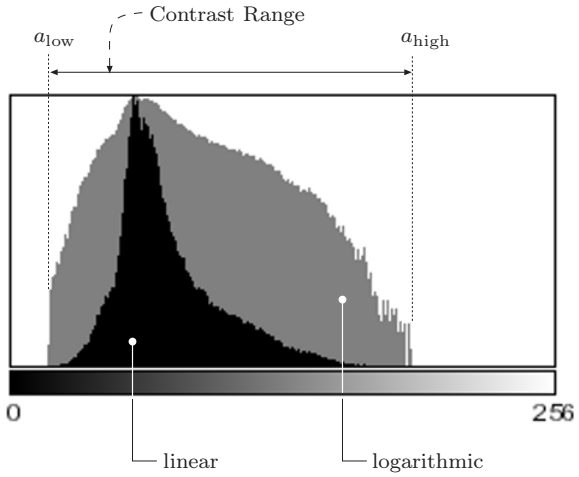


Figure 3.5 The effective intensity range. The graph depicts how often a pixel value occurs linearly (black bars) and logarithmically (gray bars). The logarithmic form makes even relatively low occurrences, which can be very important in the image, readily apparent.

3.2.1 Image Acquisition

Exposure

Histograms make classic exposure problems readily apparent. As an example, a histogram where a large span of the intensity range at one end is largely unused while the other end is crowded with high-value peaks (Fig. 3.6) is representative of an improperly exposed image.

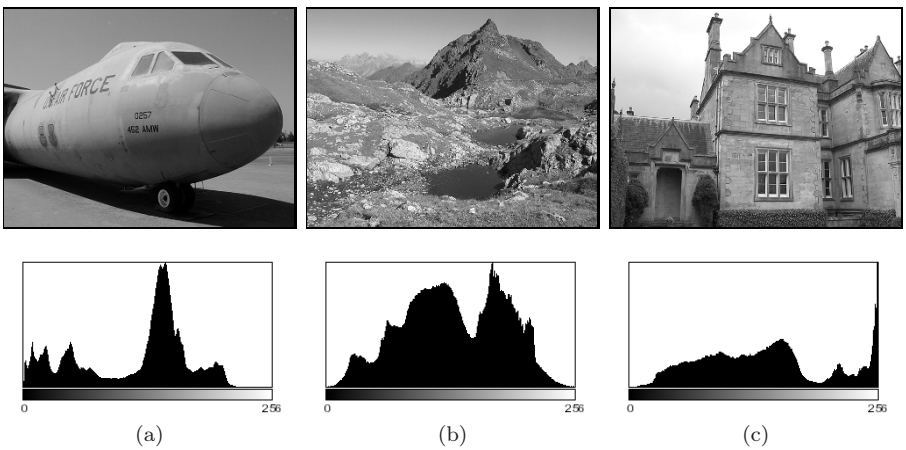


Figure 3.6 Exposure errors are readily apparent in histograms. Underexposed (a), properly exposed (b), and overexposed (c) photographs.

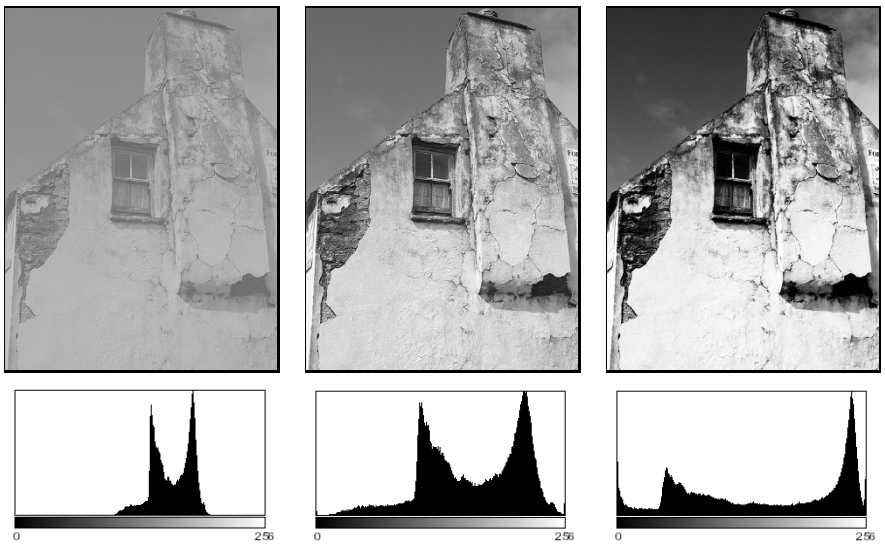


Figure 3.7 How changes in contrast affect a histogram: low contrast (a), normal contrast (b), high contrast (c).

Contrast

Contrast is understood as the range of intensity values *effectively* used within a given image, that is the difference between the image's maximum and minimum pixel values. A full-contrast image makes effective use of the entire range of available intensity values from $a = a_{\min} \ldots a_{\max} = 0 \ldots K-1$ (black to white). Using this definition, image contrast can be easily read directly from the histogram. Figure 3.7 illustrates how varying the contrast of an image affects its histogram.

Dynamic range

The dynamic range of an image is, in principle, understood as the number of *distinct* pixel values in an image. In the ideal case, the dynamic range encompasses all K usable pixel values, in which case the value range is completely utilized. When an image has an available range of contrast $a = a_{\text{low}} \ldots a_{\text{high}}$, with

$$a_{\min} < a_{\text{low}} \qquad \text{and} \qquad a_{\text{high}} < a_{\max},$$

then the maximum possible dynamic range is achieved when all the intensity values lying in this range are utilized (i. e., appear in the image; Fig. 3.8).

While the contrast of an image can be increased by transforming its existing values so that they utilize more of the underlying value range available, the dy-

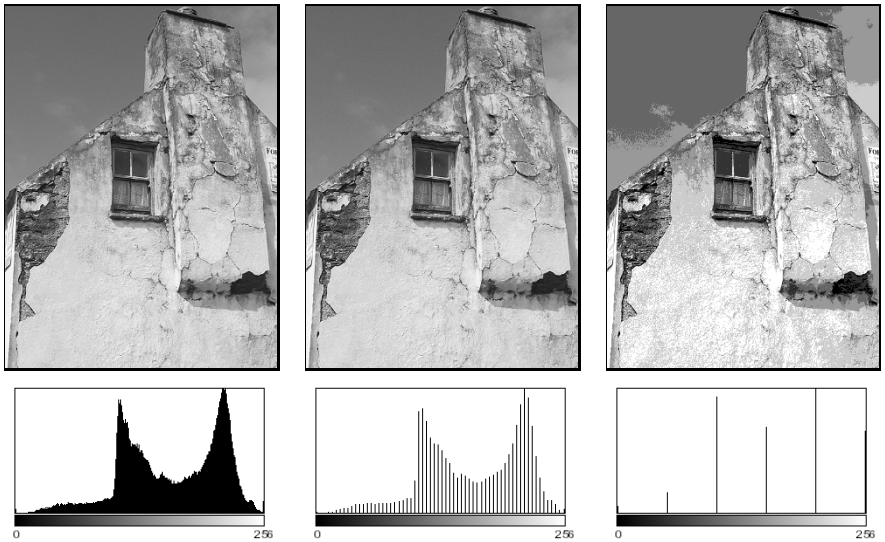


Figure 3.8 How changes in dynamic range affect a histogram: high dynamic range (a), low dynamic range with 64 intensity values (b), extremely low dynamic range with only 6 intensity values (c).

namic range of an image can only be increased by introducing artificial (that is, not originating with the image sensor) values using methods such as interpolation (see Vol. 2 [6, Sec. 10.3]). An image with a high dynamic range is desirable because it will suffer less image-quality degradation during image processing and compression. Since it is not possible to increase dynamic range after image acquisition in a practical way, professional cameras and scanners work at depths of more than 8 bits, often 12–14 bits per channel, in order to provide high dynamic range at the acquisition stage. While most output devices, such as monitors and printers, are unable to actually reproduce more than 256 different shades, a high dynamic range is always beneficial for subsequent image processing or archiving.

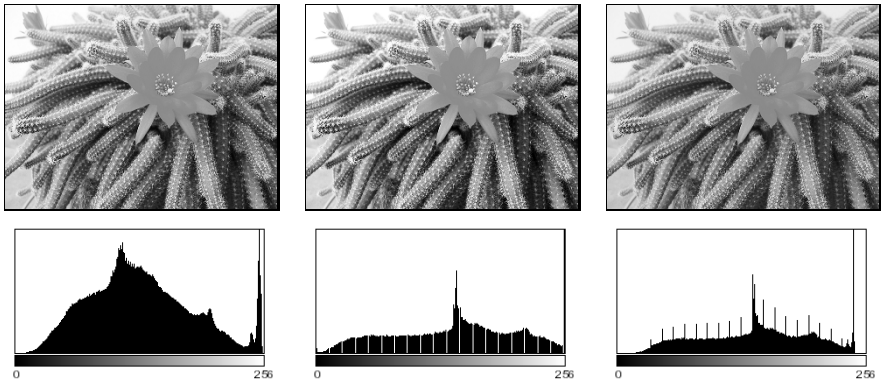
3.2.2 Image Defects

Histograms can be used to detect a wide range of image defects that originate either during image acquisition or as the result of later image processing. Since histograms always depend on the visual characteristics of the scene captured in the image, no single "ideal" histogram exists. While a given histogram may be optimal for a specific scene, it may be entirely unacceptable for another. As an example, the ideal histogram for an astronomical image would likely be very different from that of a good landscape or portrait photo. Nevertheless,

there are some general rules; for example, when taking a landscape image with a digital camera, you can expect the histogram to have evenly distributed intensity values and no isolated spikes.

Saturation

Ideally the contrast range of a sensor, such as that used in a camera, should be greater than the range of the intensity of the light that it receives from a scene. In such a case, the resulting histogram will be smooth at both ends because the light received from the very bright and the very dark parts of the scene will be less than the light received from the other parts of the scene. Unfortunately, this ideal is often not the case in reality, and illumination outside of the sensor's contrast range, arising for example from glossy highlights and especially dark parts of the scene, cannot be captured and is lost. The result is a histogram that is saturated at one or both ends of its range. The illumination values lying outside of the sensor's range are mapped to its minimum or maximum values and appear on the histogram as significant spikes at the tail ends. This typically occurs in an under- or overexposed image and is generally not avoidable when the inherent contrast range of the scene exceeds the range of the system's sensor (Fig. 3.9 (a)).



(a) (b) (c)

Figure 3.9 Effect of image capture errors on histograms: saturation of high intensities (a), histogram gaps caused by a slight increase in contrast (b), and histogram spikes resulting from a reduction in contrast (c).

Spikes and gaps

As discussed above, the intensity value distribution for an unprocessed image is generally smooth; that is, it is unlikely that isolated spikes (except for possible

saturation effects at the tails) or gaps will appear in its histogram. It is also unlikely that the count of any given intensity value will differ greatly from that of its neighbors (i. e., it is locally smooth). While artifacts like these are observed very rarely in original images, they will often be present after an image has been manipulated, for instance, by changing its contrast. Increasing the contrast (see Ch. 4) causes the histogram lines to separate from each other and, due to the discrete values, gaps are created in the histogram (Fig. 3.9 (b)). Decreasing the contrast leads, again because of the discrete values, to the merging of values that were previously distinct. This results in increases in the corresponding histogram entries and ultimately leads to highly visible spikes in the histogram (Fig. 3.9 (c)).[2]

Impacts of image compression

Image compression also changes an image in ways that are immediately evident in its histogram. As an example, during GIF compression, an image's dynamic range is reduced to only a few intensities or colors, resulting in an obvious line structure in the histogram that cannot be removed by subsequent processing (Fig. 3.10). Generally, a histogram can quickly reveal whether an image has ever been subjected to color quantization, such as occurs during conversion to a GIF image, even if the image has subsequently been converted to a full-color format such as TIFF or JPEG.

Figure 3.11 illustrates what occurs when a simple line graphic with only two gray values (128, 255) is subjected to a compression method such as JPEG, that is not designed for line graphics but instead for natural photographs. The histogram of the resulting image clearly shows that it now contains a large number of gray values that were not present in the original image, resulting in a poor-quality image[3] that appears dirty, fuzzy, and blurred.

3.3 Computing Histograms

Computing the histogram of an 8-bit grayscale image containing intensity values between 0 and 255 is a simple task. All we need is a set of 256 counters, one for each possible intensity value. First, all counters are initialized to zero.

[2] Unfortunately, these types of errors are also caused by the internal contrast "optimization" routines of some image-capture devices, especially consumer-type scanners.

[3] Using JPEG compression on images like this, for which it was not designed, is one of the most egregious of imaging errors. JPEG is designed for photographs of natural scenes with smooth color transitions, and using it to compress iconic images with large areas of the same color results in strong visual artifacts (see, for example, Fig. 1.9 on p. 19).

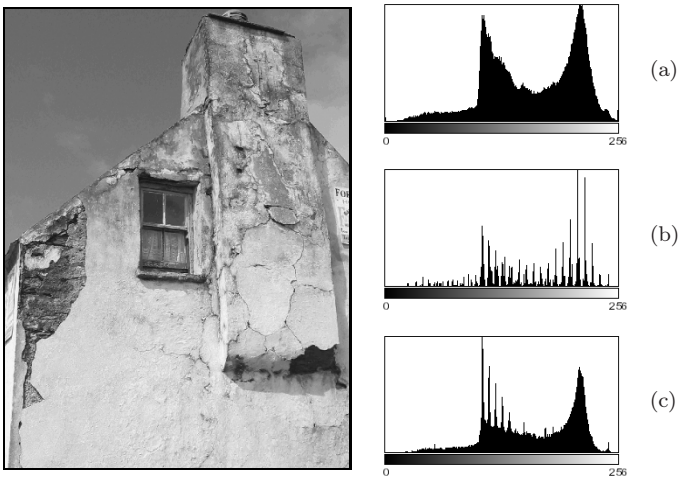


Figure 3.10 Color quantization effects resulting from GIF conversion. The original image converted to a 256 color GIF image (left). Original histogram (a) and the histogram after GIF conversion (b). When the RGB image is scaled by 50%, some of the lost colors are recreated by interpolation, but the results of the GIF conversion remain clearly visible in the histogram (c).

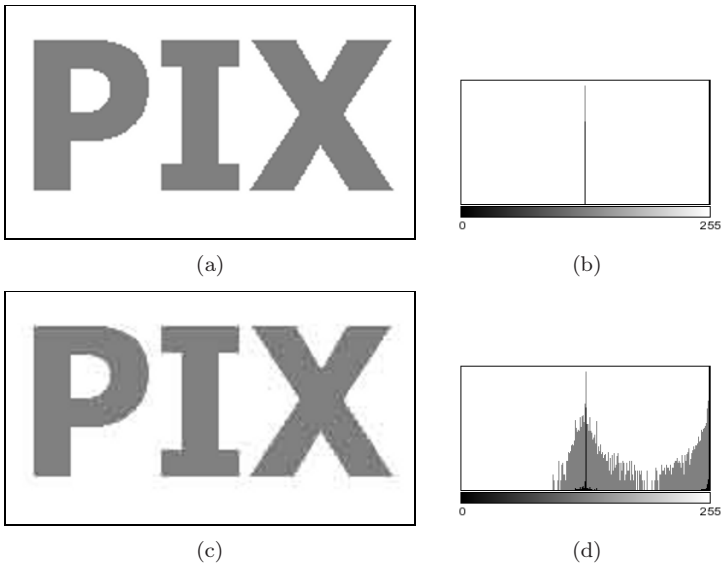


Figure 3.11 Effects of JPEG compression. The original image (a) contained only two different gray values, as its histogram (b) makes readily apparent. JPEG compression, a poor choice for this type of image, results in numerous additional gray values, which are visible in both the resulting image (c) and its histogram (d). In both histograms, the linear frequency (black bars) and the logarithmic frequency (gray bars) are shown.

```
1 public class Compute_Histogram implements PlugInFilter {
2
3     public int setup(String arg, ImagePlus img) {
4         return DOES_8G + NO_CHANGES;
5     }
6
7     public void run(ImageProcessor ip) {
8         int[] H = new int[256]; // histogram array
9         int w = ip.getWidth();
10        int h = ip.getHeight();
11
12        for (int v = 0; v < h; v++) {
13            for (int u = 0; u < w; u++) {
14                int i = ip.getPixel(u,v);
15                H[i] = H[i] + 1;
16            }
17        }
18        ... //histogram H[] can now be used
19    }
20
21 } // end of class Compute_Histogram
```

Program 3.1 ImageJ plugin for computing the histogram of an 8-bit grayscale image. The `setup()` method returns `DOES_8G + NO_CHANGES`, which indicates that this plugin requires an 8-bit grayscale image and will not alter it (line 4). In Java, all elements of a newly instantiated array (line 8) are automatically initialized, in this case to zero.

Then we iterate through the image I, determining the pixel value p at each location (u, v), and incrementing the corresponding counter by one. At the end, each counter will contain the number of pixels in the image that have the corresponding intensity value.

An image with K possible intensity values requires exactly K counter variables; for example, since an 8-bit grayscale image can contain at most 256 different intensity values, we require 256 counters. While individual counters make sense conceptually, an actual implementation would not use K individual *variables* to represent the counters but instead would use an *array* with K entries (`int[256]` in Java). In this example, the actual implementation as an array is straightforward. Since the intensity values begin at zero (like arrays in Java) and are all positive, they can be used directly as the indices $i \in [0, N-1]$ of the histogram array. Program 3.1 contains the complete Java source code for computing a histogram within the `run()` method of an ImageJ plugin.

At the start of Prog. 3.1, the array `H` of type `int[]` is created (line 8) and its elements are automatically initialized[4] to `0`. It makes no difference, at least in terms of the final result, whether the array is traversed in row or column

[4] In Java, arrays of primitives such as `int`, `double` are initialized at creation to `0` in the case of integer types or `0.0` for floating-point types, while arrays of objects are initialized to `null`.

order, as long as all pixels in the image are visited exactly once. In contrast to Prog. 2.1, in this example we traverse the array in the standard row-first order such that the outer `for` loop iterates over the *vertical* coordinates v and the inner loop over the *horizontal* coordinates u.[5] Once the histogram has been calculated, it is available for further processing steps or for being displayed.

Of course, histogram computation is already implemented in ImageJ and is available via the method `getHistogram()` for objects of the class `ImageProcessor`. If we use this built-in method, the `run()` method of Prog. 3.1 can be simplified to

```
public void run(ImageProcessor ip) {
    int[] H = ip.getHistogram();  // built-in ImageJ method
    ... // histogram H[] can now be used
}
```

3.4 Histograms of Images with More than 8 Bits

Normally histograms are computed in order to visualize the image's distribution on the screen. This presents no problem when dealing with images having $2^8 = 256$ entries, but when an image uses a larger range of values, for instance 16- and 32-bit or floating-point images (see Table 1.1), then the growing number of necessary histogram entries makes this no longer practical.

3.4.1 Binning

Since it is not possible to represent each intensity value with its own entry in the histogram, we will instead let a given entry in the histogram represent a *range* of intensity values. This technique is often referred to as "binning" since you can visualize it as collecting a range of pixel values in a container such as a bin or bucket. In a binned histogram of size B, each bin $\mathsf{h}(j)$ contains the number of image elements having values within the interval $a_j \le a < a_{j+1}$, and therefore (analogous to Eqn. (3.1))

$$\mathsf{h}(j) = \text{card}\,\{(u,v) \mid a_j \le I(u,v) < a_{j+1}\}, \quad \text{for } 0 \le j < B. \tag{3.2}$$

Typically the range of possible values in B is divided into bins of equal size $k_B = K/B$ such that the starting value of the interval j is

$$a_j = j \cdot \frac{K}{B} = j \cdot k_B.$$

[5] In this way, image elements are traversed in exactly the same way that they are laid out in computer memory, resulting in more efficient memory access and with it the possibility of increased performance, especially when dealing with larger images (see also Appendix B, p. 242).

3.4.2 Example

In order to create a typical histogram containing $B = 256$ entries from a 14-bit image, you would divide the available value range if $j = 0 \ldots 2^{14} - 1$ into 256 equal intervals, each of length $k_B = 2^{14}/256 = 64$, so that $a_0 = 0$, $a_1 = 64$, $a_2 = 128$, ... $a_{255} = 16{,}320$ and $a_{256} = a_B = 2^{14} = 16{,}384 = K$. This results in the following mapping from the pixel values to the histogram bins $\mathsf{h}(0) \ldots \mathsf{h}(255)$:

$$\begin{array}{lcrcr}
\mathsf{h}(0) & \leftarrow & 0 & \leq I(u,v) < & 64 \\
\mathsf{h}(1) & \leftarrow & 64 & \leq I(u,v) < & 128 \\
\mathsf{h}(2) & \leftarrow & 128 & \leq I(u,v) < & 192 \\
\vdots & & \vdots & \vdots & \vdots \\
\mathsf{h}(j) & \leftarrow & a_j & \leq I(u,v) < & a_{j+1} \\
\vdots & & \vdots & \vdots & \vdots \\
\mathsf{h}(255) & \leftarrow & 16320 & \leq I(u,v) < & 16384
\end{array}$$

3.4.3 Implementation

If, as in the above example, the value range $0 \ldots K-1$ is divided into equal length intervals $k_B = K/B$, there is naturally no need to use a mapping table to find a_j since for a given pixel value $a = I(u,v)$ the correct histogram element j is easily computed. In this case, it is enough to simply divide the pixel value $I(u,v)$ by the interval length k_B; that is,

$$\frac{I(u,v)}{k_B} = \frac{I(u,v)}{K/B} = \frac{I(u,v) \cdot B}{K}\,. \tag{3.3}$$

As an index to the appropriate histogram bin $\mathsf{h}(j)$, we require an integer value

$$j = \left\lfloor \frac{I(u,v) \cdot B}{K} \right\rfloor, \tag{3.4}$$

where $\lfloor \cdot \rfloor$ denotes the *floor* function.[6] A Java method for computing histograms by "linear binning" is given in Prog. 3.2. Note that all the computations from Eqn. (3.4) are done with integer numbers without using any floating-point operations. Also there is no need to explicitly call the *floor* function because the expression

```
a * B / K
```

in line 11 uses integer division and in Java the fractional result of such an operation is truncated, which is equivalent to applying the floor function (assuming

```
1  int[] binnedHistogram(ImageProcessor ip) {
2    int K = 256; // number of intensity values
3    int B = 32; // size of histogram, must be defined
4    int[] H = new int[B]; // histogram array
5    int w = ip.getWidth();
6    int h = ip.getHeight();
7
8    for (int v = 0; v < h; v++) {
9      for (int u = 0; u < w; u++) {
10       int a = ip.getPixel(u, v);
11       int i = a * B / K; // integer operations only!
12       H[i] = H[i] + 1;
13     }
14   }
15   // return binned histogram
16   return H;
17 }
```

Program 3.2 Histogram computation using "binning" (Java method). Example of computing a histogram with $B = 32$ bins for an 8-bit grayscale image with $K = 256$ intensity levels. The method `binnedHistogram()` returns the histogram of the image object `ip` passed to it as an `int` array of size B.

positive arguments).[7] The binning method can also be applied, in a similar way, to floating-point images.

3.5 Color Image Histograms

When referring to histograms of color images, typically what is meant is a histogram of the image intensity (luminance) or of the individual color channels. Both of these variants are supported by practically every image-processing application and are used to objectively appraise the image quality, especially directly after image acquisition.

3.5.1 Intensity Histograms

The intensity or *luminance* histogram $\mathsf{h}_{\mathrm{Lum}}$ of a color image is nothing more than the histogram of the corresponding grayscale image, so naturally all aspects of the preceding discussion also apply to this type of histogram. The grayscale image is obtained by computing the luminance of the individual channels of the color image. When computing the luminance, it is not sufficient to simply average the values of each color channel; instead, a weighted sum that

[6] $\lfloor x \rfloor$ rounds x down to the next whole number (see Appendix A, p. 233).
[7] For a more detailed discussion, see the section on integer division in Java in Appendix B (p. 237).

takes into account color perception theory should be computed. This process is explained in detail in Chapter 8 (p. 202).

3.5.2 Individual Color Channel Histograms

Even though the luminance histogram takes into account all color channels, image errors appearing in single channels can remain undiscovered. For example, the luminance histogram may appear clean even when one of the color channels is oversaturated. In RGB images, the blue channel contributes only a small amount to the total brightness and so is especially sensitive to this problem.

Component histograms supply additional information about the intensity distribution within the individual color channels. When computing component histograms, each color channel is considered a separate intensity image and each histogram is computed independently of the other channels. Figure 3.12 shows the luminance histogram $\mathsf{h}_{\mathrm{Lum}}$ and the three component histograms h_{R}, h_{G}, and h_{B} of a typical RGB color image. Notice that saturation problems in all three channels (red in the upper intensity region, green and blue in the lower regions) are obvious in the component histograms but not in the luminance histogram. In this case it is striking, and not at all atypical, that the three component histograms appear completely different from the corresponding luminance histogram $\mathsf{h}_{\mathrm{Lum}}$ (Fig. 3.12 (b)).

3.5.3 Combined Color Histograms

Luminance histograms and component histograms both provide useful information about the lighting, contrast, dynamic range, and saturation effects relative to the individual color components. It is important to remember that they provide no information about the distribution of the actual *colors* in the image because they are based on the individual color channels and not the combination of the individual channels that forms the color of an individual pixel. Consider, for example, when h_{R}, the component histogram for the red channel, contains the entry

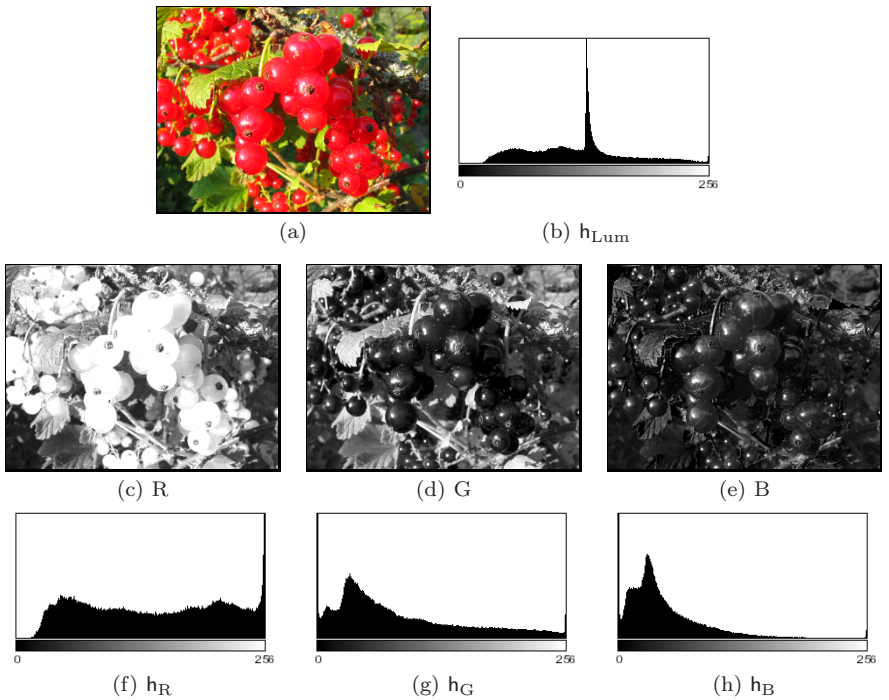
$$\mathsf{h}_{\mathrm{R}}(200) = 24.$$

Then it is only known that the image has 24 pixels that have a red intensity value of 200. The entry does not tell us anything about the green and blue values of those pixels, which could be any valid value ($*$); that is,

$$(r, g, b) = (200, *, *).$$

Suppose further that the three component histograms included the following entries:

$$\mathsf{h}_{\mathrm{R}}(50) = 100, \quad \mathsf{h}_{\mathrm{G}}(50) = 100, \quad \mathsf{h}_{\mathrm{B}}(50) = 100.$$



(a) (b) $\mathsf{h}_{\mathrm{Lum}}$ (c) R (d) G (e) B (f) h_{R} (g) h_{G} (h) h_{B}

Figure 3.12 Histograms of an RGB color image: original image (a), luminance histogram $\mathsf{h}_{\mathrm{Lum}}$ (b), RGB color components as intensity images (c–e), and the associated component histograms h_{R}, h_{G}, h_{B} (f–h). The fact that all three color channels have saturation problems is only apparent in the individual component histograms. The spike in the distribution resulting from this is found in the middle of the luminance histogram (b).

Could we conclude from this that the image contains 100 pixels with the color combination

$$(r, g, b) = (50, 50, 50)$$

or that this color occurs at all? In general, no, because there is no way of ascertaining from these data if there exists a pixel in the image in which all three components have the value 50. The only thing we could really say is that the color value $(50, 50, 50)$ can occur at most 100 times in this image.

So, although conventional (intensity or component) histograms of color images depict important properties, they do not really provide any useful information about the composition of the actual colors in an image. In fact, a collection of color images can have very similar component histograms and still contain entirely different colors. This leads to the interesting topic of the *combined* histogram, which uses statistical information about the combined color components in an attempt to determine if two images are roughly similar in their color composition. Features computed from this type of histogram often

form the foundation of color-based image retrieval methods. We will return to this topic in Chapter 8, where we will explore color images in greater detail.

3.6 Cumulative Histogram

The cumulative histogram, which is derived from the ordinary histogram, is useful when performing certain image operations involving histograms; for instance, histogram equalization (see Sec. 4.5). The cumulative histogram H is defined as

$$\mathsf{H}(i) = \sum_{j=0}^{i} \mathsf{h}(j) \quad \text{for } 0 \le i < K. \tag{3.5}$$

A particular value $\mathsf{H}(i)$ is thus the sum of all the values $\mathsf{h}(j)$, with $j \le i$, in the original histogram. Alternatively, we can define H recursively (as implemented in Prog. 4.2 on p. 66):

$$\mathsf{H}(i) = \begin{cases} \mathsf{h}(0) & \text{for } i = 0 \\ \mathsf{H}(i-1) + \mathsf{h}(i) & \text{for } 0 < i < K. \end{cases} \tag{3.6}$$

The cumulative histogram $\mathsf{H}(i)$ is a monotonically increasing function with a maximum value

$$\mathsf{H}(K-1) = \sum_{j=0}^{K-1} \mathsf{h}(j) = M \cdot N; \tag{3.7}$$

that is, the total number of pixels in an image of width M and height N. Figure 3.13 shows a concrete example of a cumulative histogram.

The cumulative histogram is useful not primarily for viewing but as a simple and powerful tool for capturing statistical information from an image. In particular, we will use it in the next chapter to compute the parameters for several common point operations (see Sections 4.4–4.6).

3.7 Exercises

Exercise 3.1

In Prog. 3.2, `B` and `K` are constants. Consider if there would be an advantage to computing the value of `B/K` outside of the loop, and explain your reasoning.

Exercise 3.2

Develop an ImageJ plugin that computes the cumulative histogram of an 8-bit grayscale image and displays it as a new image, similar to $\mathsf{H}(i)$ in Fig. 3.13.

Hint: Use the `ImageProcessor` method `int[] getHistogram()` to retrieve